

09

Lecture 09

CPU Scheduling Evaluation & Advanced Topics

by - Brijesh Shukla
C07: Operating Systems

The Scheduler's Dilemma

You're the OS scheduler. Five processes arrive:

- Emergency System Update (2 seconds, CRITICAL)
- Student's 10-hour Rendering (just started)
- Video Call (needs CPU every 50ms)
- File Download (quick bursts between waits)
- Background Backup (can wait, but needs to finish)

Question: Which runs first? Can one algorithm rule them all?



Quick Check – Algorithm Selection

If you had to choose ONE algorithm for a university's time-sharing system with 100 students, which would you pick and why?

- A) FCFS
- B) SJF
- C) Round Robin
- D) Priority



ANSWER: C) Round Robin - ensures fairness and good response time for all students. In a shared system, preventing any student from monopolizing CPU is crucial.

THE EVALUATION FRAMEWORK

8 Critical Metrics:

1. CPU Utilization - Keep the engine running (40-90%)
2. Throughput - How many jobs completed per time unit?
3. Turnaround Time - Start to finish duration
4. Waiting Time - Frustration in the ready queue
5. Response Time - Time to first acknowledgment (UX critical!)
6. Fairness - Everyone gets a turn
7. Predictability - Consistent performance
8. Overhead - Cost of context switches and management



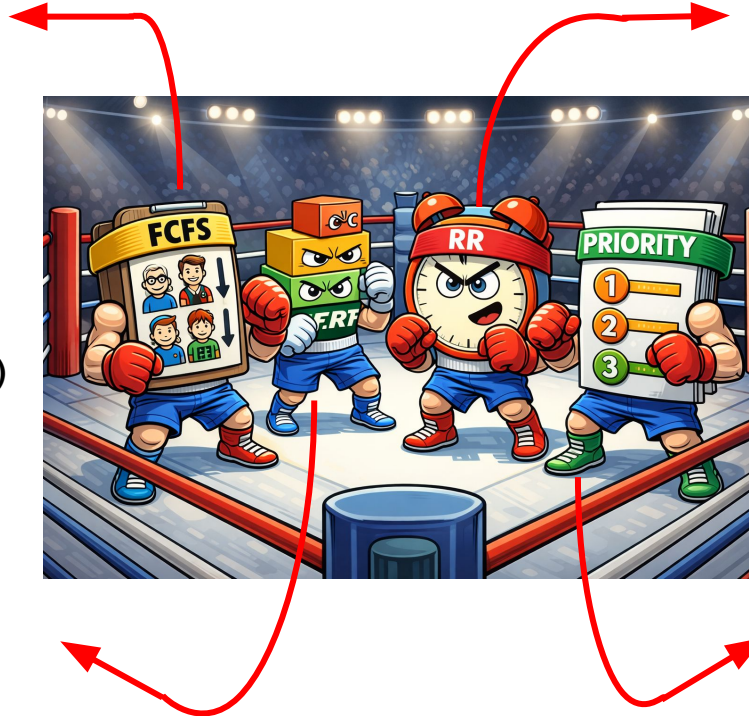
ALGORITHM QUICK RECAP

FCFS (First-Come, First-Served)

- Simple, non-preemptive
- Convoy effect problem
- Low overhead, predictable

SJF/SRTF (Shortest Job First)

- Optimal average waiting time
- Requires burst time prediction
- Starvation risk for long jobs



Round Robin

- Time quantum-based fairness
- Excellent response time
- Context switch overhead

Priority Scheduling

- Importance-based allocation
- Starvation risk (solved by aging)
- Good for real-time requirements

ALGORITHM CHAMPIONSHIP

Algorithm Comparison

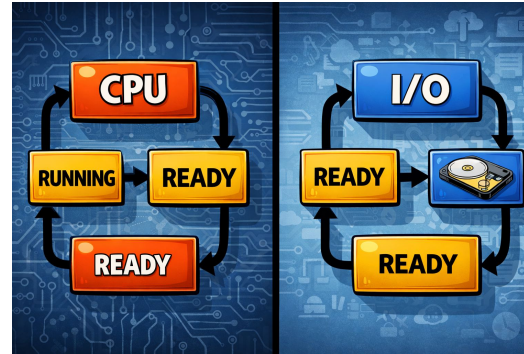
Algorithm	FCFS	SJF	SRTF	Round Robin	Priority
Response	Poor	Poor	Good	Best	Varies
Fairness	Fair	Unfair	Unfair	Best	Poor\
Overhead	Lowest	Low	High	Medium	Low-Med
Superpower	Simple & Predictable	Optimal Avg Wait	Near-Optimal	Interactive King	Real-time Ready

Key Insight: There's no clear winner! Each algorithm is like a specialized tool.

WORKLOAD CHARACTERISTICS

CPU-Bound Processes:

- Long CPU bursts, rare I/O
- Examples: Scientific computing, video encoding, 3D rendering
- Best algorithm: SJF/SRTF for efficiency



I/O-Bound Processes:

- Short CPU bursts, frequent I/O waits
- Examples: Text editors, web browsers, database queries
- Best algorithm: RR or Priority for responsiveness

Mixed Workload:

- Combination of both types
- Most realistic scenario in modern systems
- Requires balanced approach (MLFQ)

REAL-WORLD SCENARIO

The Situation: You're on a Zoom call while Netflix buffers in the background

CPU-Bound Process (Netflix decoder):

- Long CPU bursts (decoding video frames)
- Rarely waits for I/O

I/O-Bound Process (Zoom):

- Short CPU bursts (processing audio packets)
- Frequent I/O waits (network)

Wrong Choice: Zoom waits 10 seconds for its turn → Your video freezes!



Right Choice: RR or Priority → Zoom gets frequent CPU slices → Smooth call

COMPREHENSIVE EXAMPLE SETUP

The Challenge -
Same Workload, Different Stories:

Let's watch these five processes compete under different rules!

Process	Arrival	Burst	Priority
P1	0	10	3
P2	1	1	1
P3	2	2	2
P4	3	1	4
P5	4	5	2



Gantt Chart

Process	P1	P2	P3	P4	P5
Start Time	0	10	11	13	14
End Time	10	11	13	14	19

The Story:

- P2 (1 second job) waits 10 seconds behind P1
- P4 waits 13 seconds for a 1-second job!
- Avg Waiting Time: 7.4 seconds

Waiting Times: P1=0, P2=9, P3=9, P4=10, P5=9

Turnaround Times: P1=10, P2=10, P3=11, P4=11, P5=15

Verdict: "Like waiting behind someone with a full shopping cart when you only have one item"

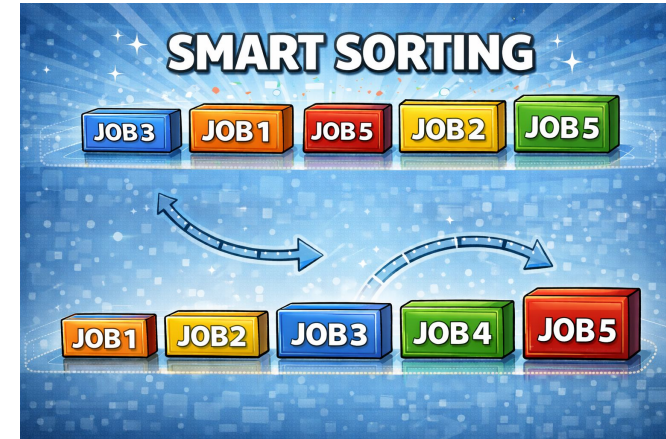
COMPREHENSIVE EXAMPLE SETUP

The Challenge -

Same Workload, Different Stories:

Let's watch these five processes compete under different rules!

Process	Arrival	Burst	Priority
P1	0	10	3
P2	1	1	1
P3	2	2	2
P4	3	1	4
P5	4	5	2



Gantt Chart

Process	P1	P2	P4	P3	P5
Execution Time	0-10	10-11	11-12	12-14	14-19

Results:

- Avg Waiting: 7.2 seconds ✓ (Better than FCFS!)
- But P1 still suffers (no preemption)
- Short jobs (P2, P4) get relief

Waiting Times: P1=0, P2=10, P3=11, P4=13, P5=15

The Insight: Mathematically proven optimal for average waiting time!

COMPREHENSIVE EXAMPLE SETUP

The Challenge -

Same Workload, Different Stories:

Let's watch these five processes compete under different rules!

Process	Arrival	Burst	Priority
P1	0	10	3
P2	1	1	1
P3	2	2	2
P4	3	1	4
P5	4	5	2



P1	P2	P3	P4	P1	P5	P1	P5	P1	P5	P1
0-2	2-3	3-5	5-6	6-8	8-10	10-12	12-14	14-16	16-17	17-19

The Magic:

- Avg Response Time: 3.4 seconds 🏆 (Best!)
- Avg Waiting: 6.4 seconds 🏆 (Also best!)
- Cost: 8 context switches (vs 0 for FCFS)

No Starvation: P1 gets multiple turns because it has a long burst. Everyone gets regular CPU access

COMPREHENSIVE EXAMPLE SETUP

The Challenge -

Same Workload, Different Stories:

Let's watch these five processes compete under different rules!

Process	Arrival	Burst	Priority
P1	0	10	3
P2	1	1	1
P3	2	2	2
P4	3	1	4
P5	4	5	2



Gantt Chart

Process	P1	P2	P3	P5	P4
Time	0-1, 9-18	1-2	2-4	4-9	18-19

The Drama:

- P2 (Priority 1) immediately preempts P1 at t=1!
- P3 (Priority 2) preempts P1 at t=2!
- P4 (Priority 4) waits 15 seconds for 1-second job! 😞
- Avg Waiting: 5.6 seconds (lowest, but unfair)

Waiting Times: P1=9, P2=0, P3=0, P4=15, P5=4

The Lesson: "Efficiency without fairness = starvation"

THE VERDICT

Algorithm Comparison

	 Avg Wait	 Avg Response	 Switches	 Winner Category
FCFS	7.4s	7.4s	0	 Simplest
SJF	7.2s	-	0	 Batch Jobs
RR (q=3)	6.4s	3.4s	8	 Interactive
Priority	5.6s	Varies	6	 Real-time

Key Insight: The "best" algorithm depends entirely on what you're optimizing for.

The Truth: NO SINGLE WINNER! Context matters.

Test Your Understanding

Which scheduling algorithm provides optimal average waiting time?

- A) FCFS
- B) SJF
- C) Round Robin
- D) Priority

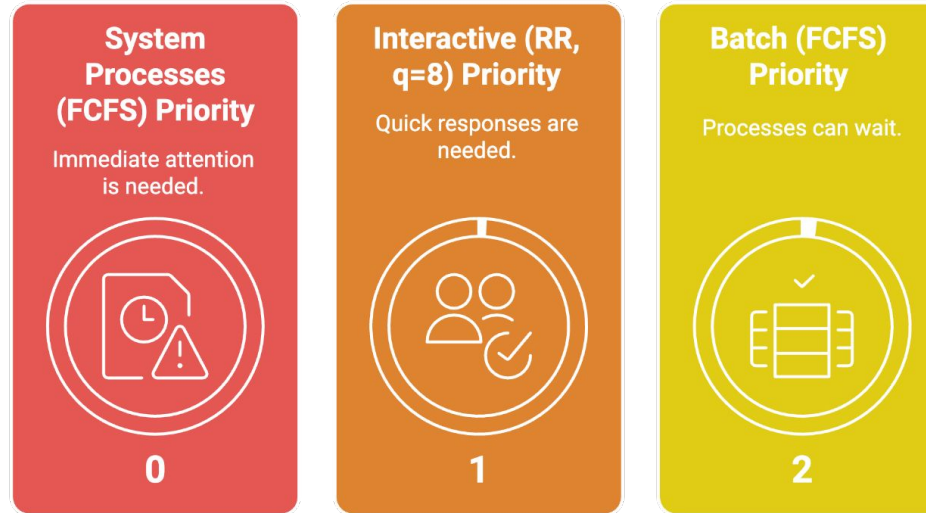
ANSWER: B) SJF

This is mathematically proven - always run the shortest job next minimizes average waiting time.



What If We Use Them ALL?

System Process Priorities



The Insight: Different processes have different needs!

The Idea: Use different algorithms for different process types!

FIXED MULTI-LEVEL QUEUES

How It Works:

1. Classify processes on arrival (system/interactive/batch)
2. Each queue has its own algorithm
3. Higher priority queues run first

Example Timeline:

- Time 0-2: System queue (P1) runs
- Time 2-10: Interactive queue (P2, P3) run with RR
- Time 10+: Batch queue (P4) finally runs

The Problem:

- ✗ Once classified, processes are stuck forever!
- ✗ What if P4 becomes interactive?
- ✗ Batch queue may starve!



THE BIG PROBLEM

How Do We Know What a Process Needs?

The Classification Challenge:

When firefox.exe starts, is it:

- Downloading files? (I/O-bound)
- Rendering video? (CPU-bound)
- Sitting idle in background?

WE CAN'T PREDICT BEHAVIOR JUST FROM THE NAME!

What if processes could **PROVE** what they need by their **BEHAVIOR**?

This leads us to the most brilliant scheduling innovation...



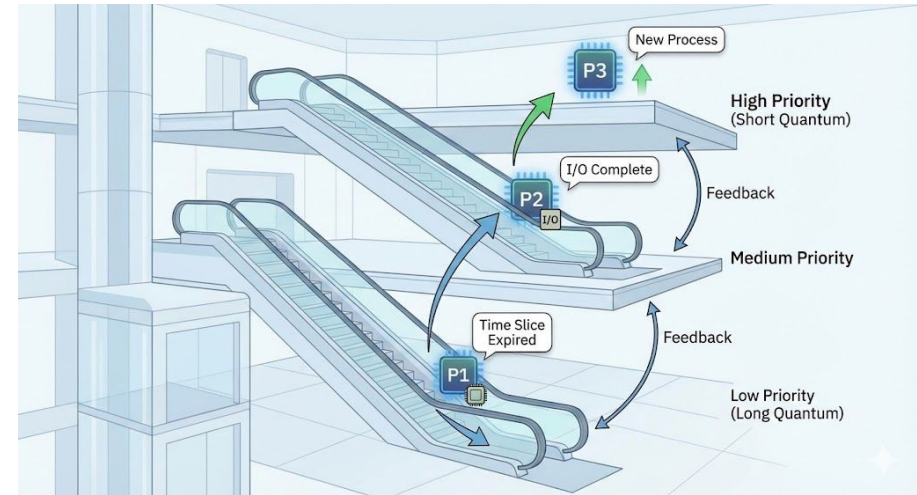
MLFQ – THE ADAPTIVE GENIUS

The Revolutionary Idea:

- 🎯 Start ALL processes at highest priority
- 🎯 If you use your full quantum → demoted (CPU-bound behavior)
- 🎯 If you yield before quantum → stay high (I/O-bound behavior)
- 🎯 Processes move dynamically based on actual behavior

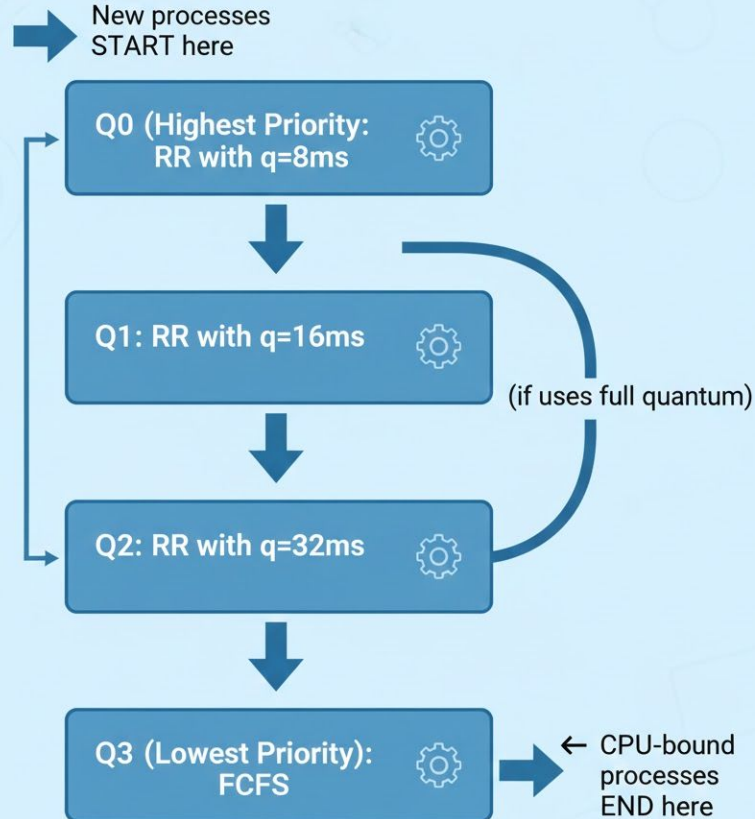
The Self-Organizing System:

- ✓ CPU-bound processes naturally sink to lower queues
- ✓ I/O-bound processes naturally float to top
- ✓ NO need to predict - let behavior decide!



This is brilliant! The system learns automatically!

MLFQ STRUCTURE



Notice:

- Quantum increases as you go down
- CPU-bound gets longer uninterrupted time
- Interactive gets frequent turns at top

MLFQ IN ACTION

Process Scheduling Timeline: P1 and P2

Time 0
P1 arrives and placed in Q0

Time 10-13
P2 runs 3ms, blocks for I/O → STAYS in Q0

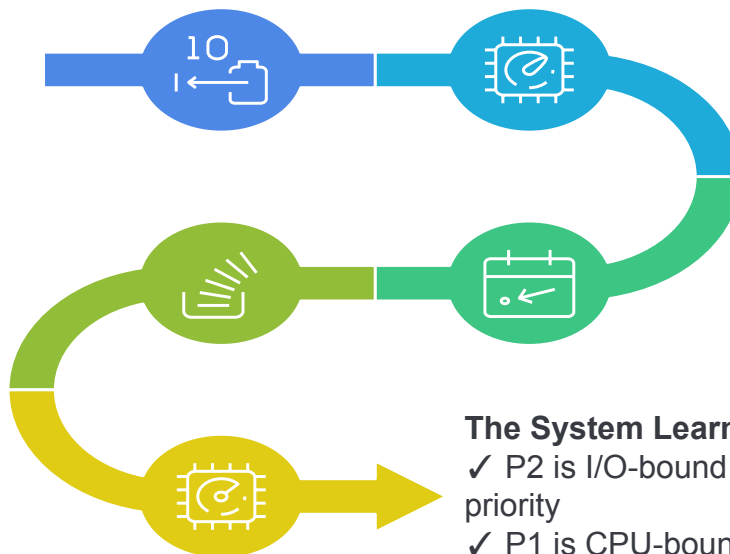
Time 13-29
P1 runs 16ms (full quantum) → demoted to Q2

Time 0-8

P1 runs 8ms (full quantum) → demoted to Q1

Time 10

P2 arrives and preempts P1



The System Learned:

- ✓ P2 is I/O-bound (didn't use full quantum) → keeps high priority
- ✓ P1 is CPU-bound (uses full quantum) → moves to lower priority

No programmer hints needed!

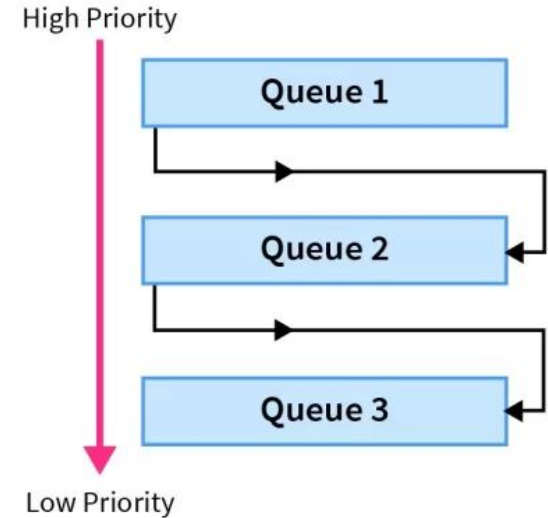
MLFQ Understanding Check

What is the main advantage of MLFQ over fixed multi-level queues?

- A) Lower overhead
- B) Simpler implementation
- C) Dynamic priority adjustment based on behavior
- D) Guaranteed optimal waiting time

ANSWER: C) Dynamic priority adjustment based on behavior

MLFQ automatically moves processes between queues based on their actual CPU usage patterns, adapting to changing behavior.



GAMING THE SYSTEM

The Problem - Smart Processes Try to Cheat:

// Gaming attempt:

```
while(work_remaining) {
    do_work_for(quantum - 1ms);
    yield_cpu(); // Give up just before demotion!
}
```

Result: Stay in high priority queue forever!

The Solutions:

1. Priority Boost:
 - Every 100ms: Move ALL processes back to Q0
 - Prevents starvation
 - Makes gaming pointless
2. Better Accounting:
 - Track TOTAL CPU time in quantum period
 - Not just current burst



MLFQ – THE MODERN STANDARD

The Superpowers:

- ✓ No prediction needed - learns from behavior
- ✓ Excellent response for interactive apps
- ✓ High throughput for CPU-bound apps
- ✓ Prevents starvation with priority boost
- ✓ Fair over the long term
- ✓ Adapts to changing process behavior

Real-World Usage:

- FreeBSD
- macOS (variant)
- Early Windows versions
- Many embedded systems



The Verdict: As close as we've come to a "universal" scheduler!

The Completely Fair Scheduler

The Philosophy Shift: What if we track FAIRNESS instead of queues?

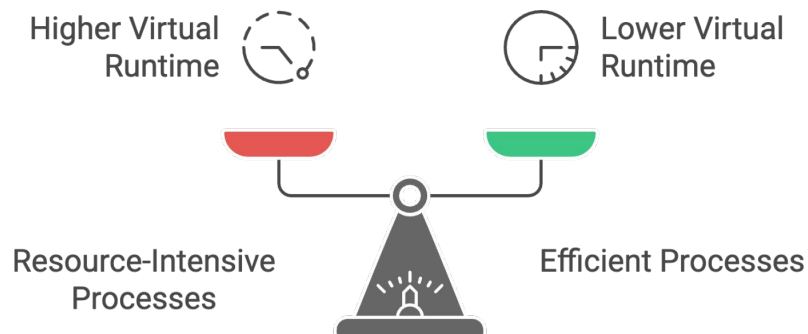
Key Concept - Virtual Runtime (vruntime)

Every process tracks: vruntime (virtual runtime)
Scheduler rule: Always run process with LOWEST vruntime

Example:

- P1 runs for 10ms $\rightarrow$ vruntime += 10
- P2 runs for 10ms $\rightarrow$ vruntime += 10
- P1 runs again $\rightarrow$ vruntime += 10 (now 20)
- Next: Schedule P2 (vruntime = 10 < 20)

The Magic: Over time, everyone gets EXACTLY fair CPU share!



CFS – THE NICE SYSTEM

Nice Values Control CPU Share:

Range: -20 (highest priority) to +19 (lowest)

Default: 0

The Math:

Lower nice value → Higher weight → vruntime increases SLOWER

Example:

P1: nice=0, weight=1024

P2: nice=5, weight=335

Both run 10ms of real time:

- P1: $\text{vruntime} += 10 \times (1024/1024) = 10$
- P2: $\text{vruntime} += 10 \times (1024/335) \approx 30$

Result: P1 gets scheduled 3× more often!



CFS Understanding Check

Linux CFS aims for:

- A) Strict priority enforcement
- B) Fair CPU share based on weights
- C) Shortest job first optimization
- D) Round robin only

ANSWER: B) Fair CPU share based on weights
CFS ensures each process gets its fair share of CPU time proportional to its weight (determined by nice values).



WINDOWS PRIORITY SCHEDULING

Priority Structure:

Level 31-16: Real-time (fixed, never change)

Level 15-1: Variable priority (dynamic)

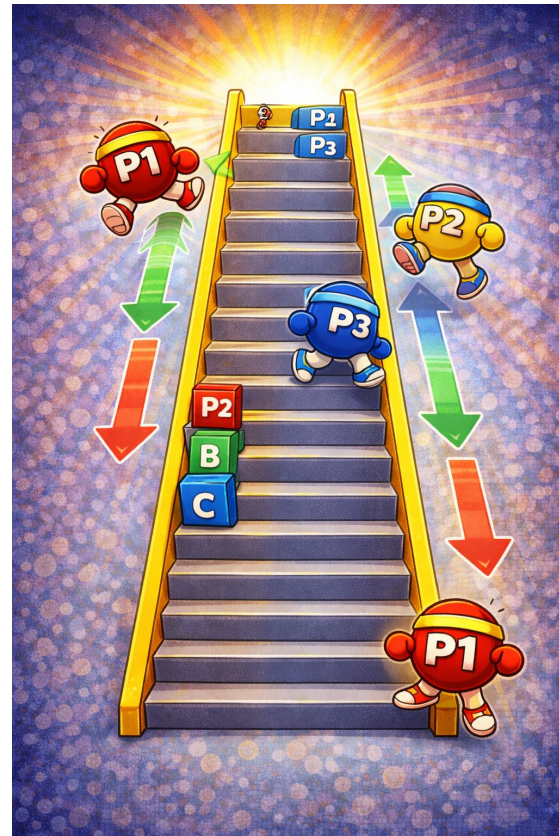
Level 0: System idle (reserved)

Dynamic Adjustments:

-  I/O completion → Temporary priority boost
-  Foreground window → Priority boost
-  CPU-bound → Gradual priority decay back to base

Philosophy: Responsiveness for user-facing tasks, throughput for background

Windows uses explicit priorities with dynamic adjustments!



MULTI-PROCESSOR REVOLUTION

The New Challenge:

Single CPU: Who runs next?

Multi-CPU: Who runs WHERE?

New Questions:

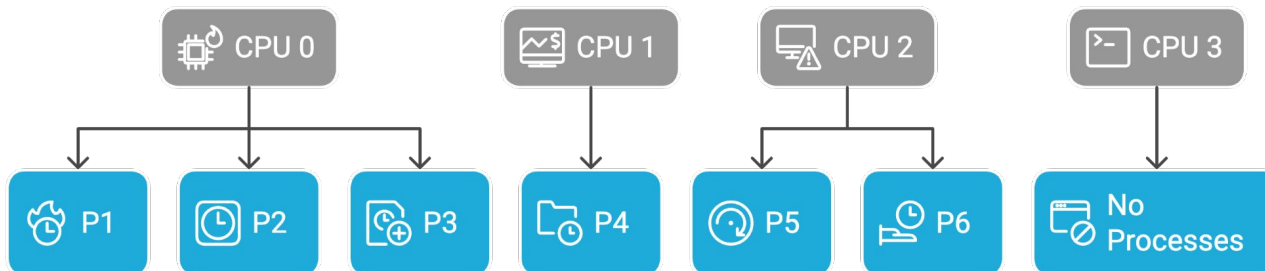
1. How to distribute load across CPUs evenly?
2. Should a process stick to one CPU?
3. What about shared caches?
4. How to synchronize schedulers?

The Opportunity: True parallelism!

4 CPUs can run 4 processes SIMULTANEOUSLY



LOAD BALANCING



Two Strategies:

Push Migration: Overloaded CPU pushes tasks away

Pull Migration: Idle CPU pulls tasks from busy ones

When to Balance?

- Periodic check (every 100ms)
- On idle detection
- On new process creation

CPU AFFINITY – THE CACHE STORY

The Cache Scenario:

P1 runs on CPU 0:

- Loads 10 MB of data into CPU 0's cache
- Runs efficiently (cache hits) ✓

P1 migrated to CPU 1:

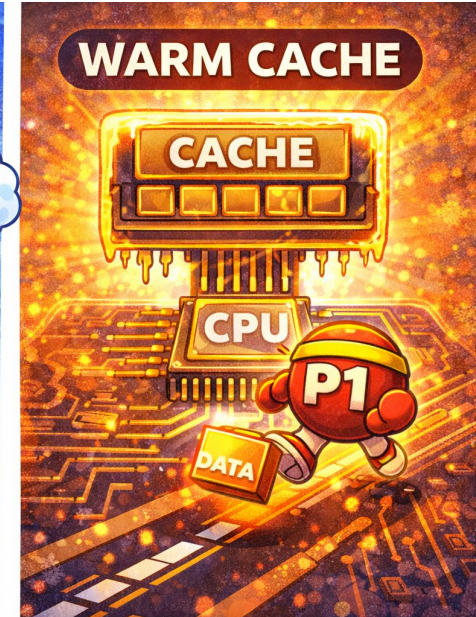
- CPU 1's cache is COLD (doesn't have P1's data)
- Must reload 10 MB from RAM
- Performance penalty: 100× slower initially! ✗

The Solution - CPU Affinity:

Soft affinity: Prefer keeping process on same CPU

Hard affinity: Lock process to specific CPU(s)

Trade-off: Load balancing vs cache performance



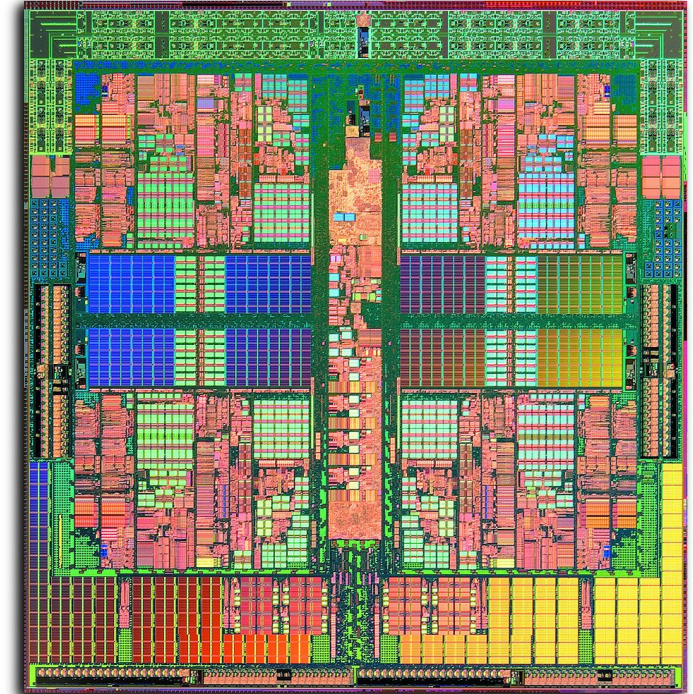
Multi-Processor Check

In multi-processor systems, CPU affinity reduces:

- A) Cache misses by keeping process on same CPU
- B) Load balancing effectiveness
- C) Preemption frequency
- D) Starvation risk

ANSWER: A) Cache misses by keeping process on same CPU

Affinity keeps the CPU cache "warm" with the process's data, avoiding the performance penalty of reloading data from main memory.



REAL-WORLD SCHEDULER SELECTION

System Type → Best Scheduler:

Web Server:

- Workload: Thousands of short requests
- Choice: MLFQ or CFS
- Reason: Fair distribution, good response time

Scientific Cluster:

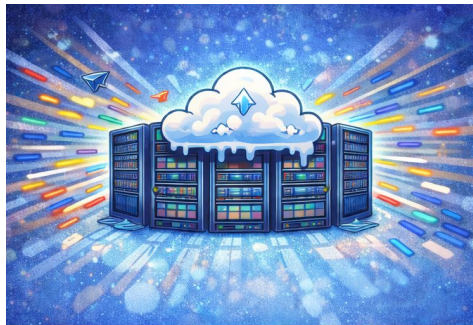
- Workload: Few long-running simulations
- Choice: SJF/SRTF or FCFS
- Reason: Maximize throughput, predictability

Desktop OS:

- Workload: Mix of everything
- Choice: MLFQ or CFS with priority boosts
- Reason: Responsiveness for user applications

Real-Time System (Missile Guidance):

- Workload: Critical timing requirements
- Choice: Priority scheduling with guarantees
- Reason: Deterministic response times



Best Algorithm Check

What is the best algorithm for hard real-time systems?

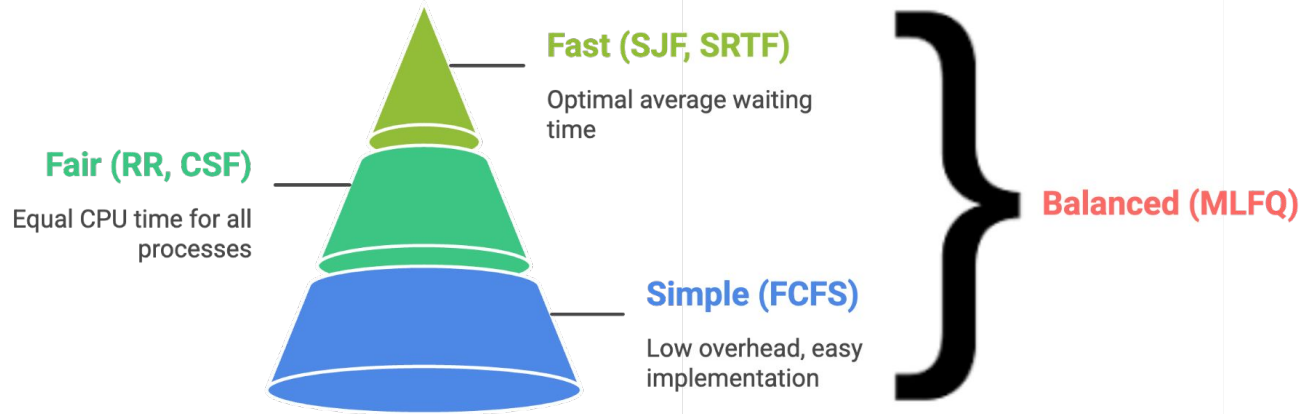
- A) FCFS
- B) Priority scheduling with guarantees
- C) Round Robin
- D) SJF

ANSWER: B) Priority scheduling with guarantees

Real-time systems need predictable, priority-based scheduling with guaranteed timing constraints. Missing a deadline could be catastrophic.



THE TRADE-OFF TRIANGLE



The Reality: Every scheduler makes trade-offs!

- FCFS: Simple + Fast, but unfair
- RR: Simple + Fair, but not fastest
- SRTF: Fair* + Fast, but complex
- MLFQ: Balanced on all three

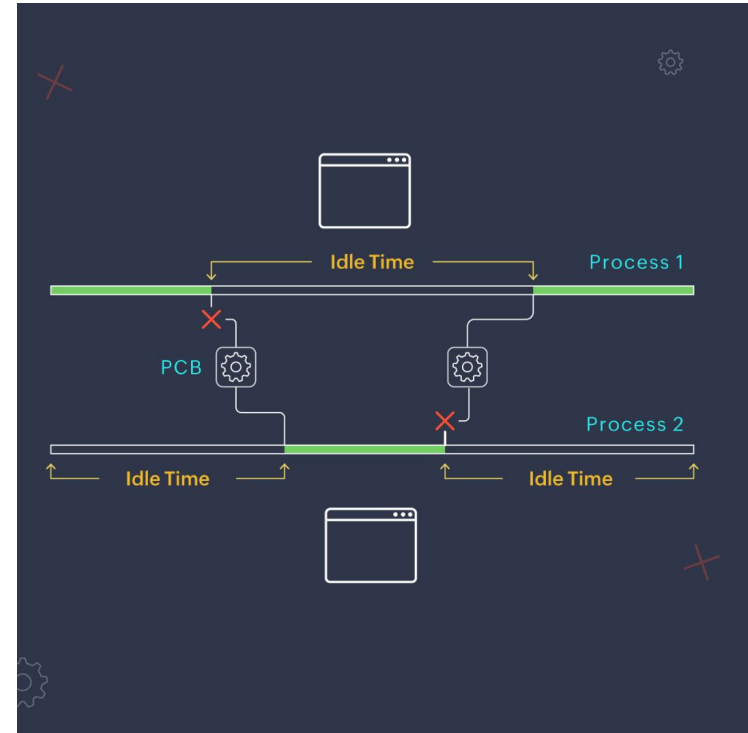
Overhead Understanding

Which algorithm has the HIGHEST overhead?

- A) FCFS
- B) SRTF
- C) Non-preemptive priority
- D) Multi-level queues

ANSWER: B) SRTF (frequent preemption)

SRTF requires frequent context switches as shorter jobs arrive, causing high overhead from saving/restoring process state.



EVOLUTION OF SCHEDULING

Scheduling Through the Decades:

1960s - Batch Systems:

FCFS, SJF → Maximize throughput

1970s - Time-Sharing:

RR → Fair interactive access

1980s - Personal Computers:

Priority + RR → Responsive GUI

1990s - Servers:

MLFQ → Handle diverse workloads

2000s - Multi-core Era:

CFS + Load Balancing → Parallel efficiency

2010s - Heterogeneous:

Big.LITTLE, GPU scheduling → Energy + performance

2020s - Cloud/AI:

Container-aware, ML-predicted → Dynamic optimization



Key Takeaways

1. Context is King
→ No universal best algorithm
2. Measure What Matters
→ Different metrics for different goals
3. Behavior Over Identity
→ MLFQ learns by watching
4. Fair $\neq$ Equal
→ CFS gives proportional fairness
5. Location Matters
→ Cache aware scheduling
6. Adapt or Stagnate
→ Static policies can't handle dynamic workloads



The Engineer's Question:

"What am I optimizing for?" determines **"Which scheduler should I use?"**

Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.